
Lecture 03

Collections and Data Structures

Basics of Internet Technologies 2

From one item to many items

Learning Objectives

- Explain why one variable is not enough for real applications.
- Use arrays and explain fixed size.
- Use `List<T>` as the main collection for changing data.
- Add, read, remove, count, index, and iterate list items.
- Find values with loops and simple conditions.
- Use `Dictionary<TKey,TValue>` for lookup by key.
- Use `HashSet<T>` for unique values.
- Use `enum` for fixed named choices.
- Explain generics at a beginner level.

Connection to Lecture 02

- Lecture 02 created an interactive menu.
- The application could add and show one current expense.
- That is better than Lecture 01, but still not enough.
- Today we keep the menu, but store many values instead of one value.

Lesson Timing Plan

- 10 min - recap and the problem with one item.
- 10 min - arrays and fixed size.
- 30 min - List<T>: Add, Count, indexing, iteration, Remove, RemoveAt.
- 15 min - searching and deleting by index.
- 15 min - parallel lists and why they are risky.
- 15 min - Dictionary and HashSet.
- 10 min - enum and generics.
- 15 min - live coding the multi-expense menu.
- 10 min - student increment and exercises.

Problem: One Item Is Not Enough

```
string expenseName = "Coffee";  
decimal expenseAmount = 4.50m;  
  
// What about Tea? Lunch? Bus ticket?
```

- A real application usually stores many records.
- Creating a new variable for every item does not scale.
- We need a structure that can hold many values together.

Bad Scaling with Separate Variables

```
string expense1 = "Coffee";  
string expense2 = "Tea";  
string expense3 = "Lunch";  
string expense4 = "Bus";
```

- This becomes ugly quickly.
- There is no easy way to loop through all expenses.
- Adding the 100th expense would require another variable name.

Data Structure Idea

- A data structure organizes values so the program can work with them.
- Different structures solve different problems.
- Today we focus on beginner structures built into C# and .NET.
- The main structure for the demo is List<T>.

Arrays Recap

```
string[] names = new string[3];  
names[0] = "Coffee";  
names[1] = "Tea";  
names[2] = "Lunch";
```

- An array stores several values of the same type.
- Indexes start at 0.
- Array length is fixed after creation.

Array Fixed Size

```
string[] names = new string[3];  
Console.WriteLine(names.Length);
```

- The array has exactly three slots.
- You cannot simply Add a fourth value to this array.
- Fixed size is useful sometimes, but not ideal for unknown user input.

Array Indexing

```
Console.WriteLine(names[0]);  
Console.WriteLine(names[1]);  
Console.WriteLine(names[2]);
```

- Index 0 is the first element.
- Index 2 is the third element.
- Index 3 would be outside a three-element array.

Array_INITIALIZER

```
string[] commands = { "add", "list", "exit" };
```

- Initializer syntax is compact for known values.
- The array length is inferred from the number of values.
- This is useful for examples and fixed option lists.

List<T>: Main Collection Today

```
List<string> names = new List<string>();  
names.Add("Coffee");  
names.Add("Tea");  
names.Add("Lunch");
```

- List<T> stores values of one type.
- A list can grow as values are added.
- T is a placeholder for the element type, such as string or decimal.

List<T> Count and Indexing

```
Console.WriteLine(names.Count);  
Console.WriteLine(names[0]);
```

- Count tells how many values are currently in the list.
- Indexing works like arrays: the first index is 0.
- Use Count, not Length, for List<T>.

List<T> with Different Types

```
List<string> expenseNames = new List<string>();  
List<decimal> expenseAmounts = new List<decimal>();  
List<int> quantities = new List<int>();
```

- The collection idea is the same.
- The element type changes what values are allowed.
- This is the first simple example of generics.

Adding Items

```
expenseNames.Add("Coffee");  
expenseAmounts.Add(4.50m);
```

- Add places a new value at the end of the list.
- After Add, Count increases by one.
- The value type must match the list type.

Reading Items by Index

```
string firstName = expenseNames[0];  
decimal firstAmount = expenseAmounts[0];
```

- Indexing is useful when the position matters.
- The index must be inside the list range.
- For display, users usually see index + 1, not the zero-based index.

Displaying User Numbers

```
for (int index = 0; index < expenseNames.Count; index++)  
{  
    int displayNumber = index + 1;  
    Console.WriteLine(displayNumber + ". " + expenseNames[index]);  
}
```

- Program indexes start at 0.
- Human-friendly numbers usually start at 1.
- Convert display number back to index when deleting.

foreach with List<T>

```
foreach (string name in expenseNames)
{
    Console.WriteLine(name);
}
```

- foreach is clear when you only need the value.
- It hides the index.
- Use for when you need index-based access to parallel lists.

Removing by Value

```
expenseNames.Remove("Tea");
```

- Remove deletes the first matching value.
- It returns whether something was removed.
- This is simple for unique values, but ambiguous when duplicates exist.

Removing by Index

```
expenseNames.RemoveAt(0);
```

- RemoveAt deletes the value at a specific index.
- Indexes after that position move left.
- Always validate the index before removing.

Parallel Lists

```
List<string> names = new List<string>();  
List<decimal> amounts = new List<decimal>();  
  
names.Add("Coffee");  
amounts.Add(4.50m);
```

- Parallel lists store related data at the same index.
- `names[0]` belongs with `amounts[0]`.
- This is useful for now, but risky if the lists get out of sync.

Why Parallel Lists Are Risky

```
names.RemoveAt(0);  
// Forgot: amounts.RemoveAt(0);
```

- Now names and amounts no longer describe the same records.
- This is a real problem students can understand before OOP.
- Lecture 05 will solve it with classes and objects.

Finding with a Loop

```
string search = "cof";  
for (int index = 0; index < names.Count; index++)  
{  
    if (names[index].ToLowerInvariant().Contains(search))  
    {  
        Console.WriteLine(names[index]);  
    }  
}
```

- Searching combines a loop and a condition.
- Contains checks whether text includes another text.
- ToLowerInvariant makes a simple case-insensitive search possible.

Delete by Display Number

```
int index = displayNumber - 1;

if (index >= 0 && index < names.Count)
{
    names.RemoveAt(index);
    amounts.RemoveAt(index);
}
```

- The user enters 1, 2, 3, ...
- The list needs indexes 0, 1, 2, ...
- Validation prevents out-of-range errors.

Summing a List

```
decimal total = 0m;  
foreach (decimal amount in amounts)  
{  
    total += amount;  
}
```

- Start with an empty total.
- Add each amount to the total.
- This is a common collection pattern.

Dictionary<TKey, TValue>

```
Dictionary<string, decimal> pricesByName = new Dictionary<string, decimal>();  
pricesByName["Coffee"] = 4.50m;  
pricesByName["Tea"] = 3.20m;
```

- A dictionary stores key-value pairs.
- The key is used for lookup.
- Keys must be unique.

Dictionary Lookup

```
if (pricesByName.ContainsKey("Tea"))  
{  
    Console.WriteLine(pricesByName["Tea"]);  
}
```

- Check that a key exists before reading it.
- Reading a missing key causes an error.
- Dictionaries are useful when lookup by key is the main operation.

TryGetValue

```
if (pricesByName.TryGetValue("Water", out decimal price))
{
    Console.WriteLine(price);
}
else
{
    Console.WriteLine("Not found");
}
```

- TryGetValue combines checking and reading.
- It returns true when the key exists.
- It avoids reading a missing key directly.

Iterating a Dictionary

```
foreach (KeyValuePair<string, decimal> pair in pricesByName)
{
    Console.WriteLine(pair.Key + " -> " + pair.Value);
}
```

- Each dictionary item has a Key and a Value.
- Order is not the main point of a dictionary.
- Use a dictionary for lookup, not for ordered display.

HashSet<T>: Unique Values

```
HashSet<string> categories = new HashSet<string>();  
categories.Add("Food");  
categories.Add("Transport");  
categories.Add("Food");
```

- A hash set stores unique values.
- Adding the same value twice does not create a duplicate.
- Use it when uniqueness matters more than order.

HashSet Example

```
Console.WriteLine(categories.Count);  
Console.WriteLine(categories.Contains("Food"));
```

- Count shows how many unique values exist.
- Contains checks whether a value is present.
- This can help with tags, categories, usernames, or visited items.

enum: Named Choices

```
enum ExpenseCategory
{
    Food = 1,
    Transport = 2,
    Study = 3,
    Other = 4
}
```

- enum gives names to fixed choices.
- It is better than magic numbers such as 1, 2, 3.
- Enums are useful for status, category, role-like labels, and modes.

Using an enum

```
ExpenseCategory category = ExpenseCategory.Food;

if (category == ExpenseCategory.Food)
{
    Console.WriteLine("Food expense");
}
```

- The code becomes easier to read.
- The compiler helps limit possible values.
- Later projects can use enums for TaskStatus, BookingStatus, or ExpenseCategory.

Generics: Beginner View

```
List<int> numbers = new List<int>();  
List<string> words = new List<string>();  
List<ExpenseCategory> categories = new List<ExpenseCategory>();
```

- Generic types use angle brackets.
- The type inside the angle brackets says what the collection stores.
- This gives type safety while reusing one collection design.

Choosing the Right Structure

Structure

- Array
- List<T>
- Dictionary<TKey,TValue>
- HashSet<T>
- enum

Use When

- Fixed number of values
- Changing number of values
- Lookup by key
- Unique values
- Fixed named choices

Live Demo: Multi-expense Menu

```
1. Add expense
2. List expenses
3. Find expense by name
4. Delete expense by number
5. Show total
6. Exit
```

- The demo now stores multiple expenses.
- `List<T>` is the main storage mechanism.
- Find and delete combine loops, indexes, and validation.

Demo Storage

```
List<string> expenseNames = new List<string>();  
List<decimal> expenseAmounts = new List<decimal>();  
List<string> expenseCategories = new List<string>();
```

- This is intentionally not OOP yet.
- The lists are parallel and must stay synchronized.
- That weakness prepares students for classes later.

Student Project Increment

- Update the Lecture 02 project so it can store multiple objects.
- Use `List<T>` as the main collection.
- Add commands: Add, List, Find, Delete.
- Use Count, indexing, for, and foreach where appropriate.
- Validate indexes before deleting.
- Optionally use enum for status or category.
- Keep the project single-user and in-memory for now.

Student Examples

Project

- Task Manager
- Expense Tracker
- Veterinary Clinic
- Movie Collection

Multiple Items

- Tasks with title and status
- Expenses with amount and category
- Pets or visits
- Movies with title and rating

What Not To Add Yet

- Do not add JSON persistence yet.
- Do not add classes yet unless the teacher explicitly permits preview work.
- Do not add database storage.
- Do not add login or users.
- Today is about in-memory multiple items and choosing basic data structures.

Common Mistakes Today

- Using array when the number of values changes.
- Using Length on List<T> instead of Count.
- Forgetting that indexes start at 0.
- Deleting from one parallel list but not the others.
- Reading a missing dictionary key directly.
- Using Dictionary when the order of display is the main requirement.

Exercises

- Add a category filter command to the demo.
- Add a command that shows the largest expense.
- Add a `HashSet<string>` to collect unique categories.
- Create an enum for three statuses in your own project.
- Write a method that prints all items from a `List<string>`.
- Write a method that validates a delete number before `RemoveAt`.

Review Questions

- Why is one variable per item a bad approach?
- What is the difference between array Length and list Count?
- When should you use for instead of foreach?
- Why are parallel lists risky?
- What problem does Dictionary solve?
- What problem does HashSet solve?
- Why is enum better than magic numbers?
- What does the T in List<T> represent?

Summary

- Collections let programs work with many values.
- Arrays are fixed size.
- `List<T>` is the main beginner collection for changing data.
- `Dictionary<TKey,TValue>` supports lookup by key.
- `HashSet<T>` supports uniqueness.
- `enum` gives readable names to fixed choices.
- The student project now supports Add, List, Find, and Delete for multiple objects.